Lab program – 1

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS

```
#include <stdio.h>
int main() {
   int n, i;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   int pid[n], at[n], bt[n], ct[n], tat[n], wt[n];
   int current_time = 0;
   // Input
   for(i = 0; i < n; i++) {
      printf("\nEnter Process ID: ");
      scanf("%d", &pid[i]);
      printf("Enter Arrival Time: ");
      scanf("%d", &at[i]);
      printf("Enter Burst Time: ");
      scanf("%d", &bt[i]);
   }
   // FCFS Calculation
   for(i = 0; i < n; i++) {
      if(current_time < at[i])
         current_time = at[i];
      ct[i] = current_time + bt[i];
      tat[i] = ct[i] - at[i];
      wt[i] = tat[i] - bt[i];
```

```
        current_time = ct[i];
    }
    // Output
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    return 0;
}
```

OUTPUT:

```
Enter number of processes: 3

Enter Process ID: 1
Enter Arrival Time: 4
Enter Burst Time: 6

Enter Process ID: 2
Enter Arrival Time: 5
Enter Burst Time: 6

Enter Process ID: 3
Enter Arrival Time: 5
Enter Burst Time: 6

PID     AT      BT      CT      TAT     WT
1       4       6       10      6       0
2       5       6       16      11      5
3       5       6       22      17      11
```

b) SJF non preemptive

```
#include <stdio.h>
int main()
{
    int n, i, j;
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
int pid[n], at[n], bt[n], ct[n], tat[n], wt[n];
int completed[n];
for(i = 0; i < n; i++)
    completed[i] = 0;
// Input
for(i = 0; i < n; i++)
{
    printf("\nEnter Process ID: ");
    scanf("%d", &pid[i]);
    printf("Enter Arrival Time: ");
    scanf("%d", &at[i]);
    printf("Enter Burst Time: ");
    scanf("%d", &bt[i]);
}
int current_time = 0, done = 0;
while(done < n)
{
    int idx = -1;
    int min_bt = 9999;
    // Find process with smallest burst time
    for(i = 0; i < n; i++)
    {
        if(at[i] <= current_time && completed[i] == 0)
        {
            if(bt[i] < min_bt)
            {
                min_bt = bt[i];
```

```
                idx = i;
            }
        }
    }
    if(idx == -1)
    {
        current_time++;
    }
    else
    {
        ct[idx] = current_time + bt[idx];
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];
        current_time = ct[idx];
        completed[idx] = 1;
        done++;
    }
  }
  // Output
  printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
  for(i = 0; i < n; i++)
  {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
    pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
  }
  return 0;
}
```

OUTPUT:

```
Enter number of processes: 3

Enter Process ID: 1
Enter Arrival Time: 3
Enter Burst Time: 2

Enter Process ID: 2
Enter Arrival Time: 4
Enter Burst Time: 4

Enter Process ID: 3
Enter Arrival Time: 4
Enter Burst Time: 6

PID     AT      BT      CT      TAT     WT
1       3       2       5       2       0
2       4       4       9       5       1
3       4       6       15      11      5
```

c) SJF preemptive

```
#include <stdio.h>
int main()
{
   int n, i;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   int pid[n], at[n], bt[n], rt[n];
   int ct[n], tat[n], wt[n];
   // Input
   for(i = 0; i < n; i++)
   {
      printf("\nEnter Process ID: ");
      scanf("%d", &pid[i]);

      printf("Enter Arrival Time: ");
```

```
    scanf("%d", &at[i]);
    printf("Enter Burst Time: ");
    scanf("%d", &bt[i]);
    rt[i] = bt[i]; // remaining time = burst time
}
int completed = 0, current_time = 0;
while(completed < n)
{
    int idx = -1;
    int min_rt = 9999;
    // find process with smallest remaining time
    for(i = 0; i < n; i++)
    {
        if(at[i] <= current_time && rt[i] > 0)
        {
            if(rt[i] < min_rt)
            {
                min_rt = rt[i];
                idx = i;
            }
        }
    }
    if(idx == -1)
    {
        current_time++;
    }
    else
    {
        rt[idx]--;       // execute for 1 unit
```

```
            current_time++;
            if(rt[idx] == 0)
            {
                ct[idx] = current_time;
                tat[idx] = ct[idx] - at[idx];
                wt[idx] = tat[idx] - bt[idx];
                completed++;
            }
        }
    }
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    return 0;
}
```

OUTPUT:

```
Enter number of processes: 3

Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 5

Enter Process ID: 2
Enter Arrival Time: 2
Enter Burst Time: 4

Enter Process ID: 3
Enter Arrival Time: 5
Enter Burst Time: 6

PID     AT      BT      CT      TAT     WT
1       0       5       5       5       0
2       2       4       9       7       3
3       5       6       15      10      4
```

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) Priority Scheduling (Non-Preemptive)

```
#include <stdio.h>
int main() {
   int n, i, j;
   printf("Enter number of processes: ");
   scanf("%d",&n);
   int pid[n], at[n], bt[n], pr[n];
   int ct[n], tat[n], wt[n], rt[n];
   int completed[n];
   for(i=0;i<n;i++)
      completed[i]=0;
   printf("\nEnter Process ID:\n");
   for(i=0;i<n;i++)
      scanf("%d",&pid[i]);
   printf("Enter Arrival Time:\n");
   for(i=0;i<n;i++)
      scanf("%d",&at[i]);
   printf("Enter Burst Time:\n");
   for(i=0;i<n;i++)
      scanf("%d",&bt[i]);
   printf("Enter Priority (Lower number = Higher priority):\n");
   for(i=0;i<n;i++)
      scanf("%d",&pr[i]);
   int time=0, count=0;
   float total_tat=0, total_wt=0;
   while(count<n) {
```

```
int idx=-1;
int highest=999;
for(i=0;i<n;i++) {
    if(at[i]<=time && completed[i]==0) {
        if(pr[i] < highest) {
            highest = pr[i];
            idx = i;
        }
        if(pr[i]==highest) {
            if(at[i] < at[idx])
                idx=i;
        }
    }
}
if(idx != -1) {
    int start = time;
    time += bt[idx];
    ct[idx] = time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];
    rt[idx] = start - at[idx];
    total_tat += tat[idx];
    total_wt += wt[idx];
    completed[idx] = 1;
    count++;
}
else {
    time++;
}
```

```
    }
    printf("\nPriority Scheduling (Non-Preemptive)\n\n");
    printf("PID\tPR\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for(i=0;i<n;i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],pr[i],at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
    }
    printf("\nAverage Turnaround Time = %.2f", total_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt/n);
    return 0;
}
```

OUTPUT:

```
Enter number of processes: 5

Enter Process ID:
1 2 3 4 5
Enter Arrival Time:
0 3 5 6 9
Enter Burst Time:
4 5 3 6 2
Enter Priority (Lower number = Higher priority):
3 4 2 1 5

Priority Scheduling (Non-Preemptive)

PID     PR      AT      BT      CT      TAT     WT      RT
1       3       0       4       4       4       0       0
2       4       3       5       9       6       1       1
3       2       5       3       18      13      10      10
4       1       6       6       15      9       3       3
5       5       9       2       20      11      9       9

Average Turnaround Time = 8.60
Average Waiting Time = 4.60
```

b) Priority Scheduling (Preemptive)

```
#include <stdio.h>
```

```
#include <limits.h>
int main()
{
    int n;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    int pid[n], at[n], bt[n], pr[n];
    int ct[n], tat[n], wt[n], rt[n];
    int rem_bt[n], start[n];
    for(int i=0;i<n;i++)
        start[i] = -1;
    printf("Enter Process ID:\n");
    for(int i=0;i<n;i++)
        scanf("%d",&pid[i]);
    printf("Enter Arrival Time:\n");
    for(int i=0;i<n;i++)
        scanf("%d",&at[i]);
    printf("Enter Burst Time:\n");
    for(int i=0;i<n;i++)
    {
        scanf("%d",&bt[i]);
        rem_bt[i] = bt[i];
    }
    printf("Enter Priority (Lower number = Higher priority):\n");
    for(int i=0;i<n;i++)
        scanf("%d",&pr[i]);
    int completed = 0;
    int time = 0;
```

```
while(completed < n)
{
    int idx = -1;
    int highest = INT_MAX;
    for(int i=0;i<n;i++)
    {
        if(at[i] <= time && rem_bt[i] > 0)
        {
            if(pr[i] < highest)
            {
                highest = pr[i];
                idx = i;
            }
            else if(pr[i] == highest)
            {
                if(at[i] < at[idx])
                    idx = i;
            }
        }
    }

    if(idx != -1)
    {
        if(start[idx] == -1)
            start[idx] = time;

        rem_bt[idx]--;
        time++;
```

```
        if(rem_bt[idx] == 0)
        {
            ct[idx] = time;
            completed++;
        }
    }
    else
    {
        time++;
    }
}
float total_tat = 0, total_wt = 0;
for(int i=0;i<n;i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    rt[i] = start[i] - at[i];
    total_tat += tat[i];
    total_wt += wt[i];
}
printf("\nPriority Scheduling (Preemptive)\n\n");
printf("PID\tPR\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(int i=0;i<n;i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    pid[i], pr[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}
printf("\nAverage Turnaround Time = %.2f", total_tat/n);
printf("\nAverage Waiting Time = %.2f\n", total_wt/n);
```

```
    return 0;
}
```

OUTPUT:

```
Enter number of processes: 5
Enter Process ID:
1 2 3 4 5
Enter Arrival Time:
0 5 2 4 3
Enter Burst Time:
3 4 3 2 5
Enter Priority (Lower number = Higher priority):
5 3 2 4 1

Priority Scheduling (Preemptive)

PID     PR      AT      BT      CT      TAT     WT      RT
1       5       0       3       17      17      14      0
2       3       5       4       14      9       5       5
3       2       2       3       10      8       5       0
4       4       4       2       16      12      10      10
5       1       3       5       8       5       0       0

Average Turnaround Time = 10.20
Average Waiting Time = 6.80
```

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

c) RoundRobin

```
#include <stdio.h>
int main()
{
    int n, tq;
    printf("Enter number of processes: ");
    scanf("%d",&n)
    int pid[n], at[n], bt[n];
    int ct[n], tat[n], wt[n], rt[n];
```

```
int rem_bt[n], start[n];
int queue[100], front=0, rear=0;
int visited[n];
for(int i=0;i<n;i++)
{
   start[i] = -1;
   visited[i] = 0;
}
printf("Enter Process ID:\n");
for(int i=0;i<n;i++)
   scanf("%d",&pid[i]);
printf("Enter Arrival Time:\n");
for(int i=0;i<n;i++)
   scanf("%d",&at[i]);
printf("Enter Burst Time:\n");
for(int i=0;i<n;i++)
{
   scanf("%d",&bt[i]);
   rem_bt[i] = bt[i];
}
printf("Enter Time Quantum: ");
scanf("%d",&tq);
int time = 0, completed = 0;
while(completed < n)
{
   for(int i=0;i<n;i++)
   {
      if(at[i] <= time && visited[i]==0)
      {
```

```
        queue[rear++] = i;
        visited[i] = 1;
    }
}
if(front == rear)
{
    time++;
    continue;
}
int idx = queue[front++];
if(start[idx] == -1)
    start[idx] = time;
if(rem_bt[idx] > tq)
{
    time += tq;
    rem_bt[idx] -= tq;
}
else
{
    time += rem_bt[idx];
    rem_bt[idx] = 0;
    ct[idx] = time;
    completed++;
}
for(int i=0;i<n;i++)
{
    if(at[i] <= time && visited[i]==0)
    {
        queue[rear++] = i;
```

```
            visited[i] = 1;
        }
    }
    if(rem_bt[idx] > 0)
        queue[rear++] = idx;
}
float total_tat = 0, total_wt = 0;
for(int i=0;i<n;i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    rt[i] = start[i] - at[i];
    total_tat += tat[i];
    total_wt += wt[i];
}
printf("\nRound Robin Scheduling\n\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(int i=0;i<n;i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    pid[i],at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
}
printf("\nAverage Turnaround Time = %.2f", total_tat/n);
printf("\nAverage Waiting Time = %.2f\n", total_wt/n);
return 0;
}
```

OUTPUT:

```
Enter number of processes: 5
Enter Process ID:
1 2 3 4 5
Enter Arrival Time:
0 3 2 4 5
Enter Burst Time:
5 3 6 3 1
Enter Time Quantum: 2

Round Robin Scheduling

PID     AT      BT      CT      TAT     WT      RT
1       0       5       14      14      9       0
2       3       3       15      12      9       3
3       2       6       18      16      10      0
4       4       3       16      12      9       4
5       5       1       13      8       7       7

Average Turnaround Time = 12.40
Average Waiting Time = 8.80
```

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories –system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>

#define MAX 20

#define TQ 2

typedef struct {

   int pid, at, bt, rt;

   int ct, tat, wt;

   int type; // 1-System, 2-User

   int done;

} Process;

int main() {

   int n;

   Process p[MAX];
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
for(int i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("\nProcess %d\n", i+1);
    printf("Arrival Time: ");
    scanf("%d", &p[i].at);
    printf("Burst Time: ");
    scanf("%d", &p[i].bt);
    printf("Type (1-System, 2-User): ");
    scanf("%d", &p[i].type);
    p[i].rt = p[i].bt;
    p[i].done = 0;
}
int time = 0, completed = 0;
while(completed < n) {
    int executed = 0;
    // 1. System Queue (Round Robin with preemption)
    for(int i = 0; i < n; i++) {
        if(p[i].type == 1 && p[i].rt > 0 && p[i].at <= time) {
            int exec = (p[i].rt > TQ) ? TQ : p[i].rt;
            p[i].rt -= exec;
            time += exec;
            // Check if finished
            if(p[i].rt == 0) {
                p[i].ct = time;
                p[i].done = 1;
                completed++;
            }
```

```
            executed = 1;
        }
    }
    if(executed) continue;
    // 2. User Queue (FCFS but PREEMPT if system arrives)
    for(int i = 0; i < n; i++) {
        if(p[i].type == 2 && p[i].rt > 0 && p[i].at <= time) {
            // execute 1 unit at a time to allow preemption
            p[i].rt--;
            time++;
            // Check completion
            if(p[i].rt == 0) {
                p[i].ct = time;
                p[i].done = 1;
                completed++;

            executed = 1;
            break; // FCFS
        }
    }
    // If nothing executed → jump time forward
    if(!executed) {
        time++;
    }
}
// Calculate TAT & WT
float total_tat = 0, total_wt = 0;
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
    for(int i = 0; i < n; i++) {
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
        total_tat += p[i].tat;
        total_wt += p[i].wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt,
            p[i].ct, p[i].tat, p[i].wt);
    }
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
    printf("Average Waiting Time = %.2f\n", total_wt / n);
    return 0;
}
```

OUTPUT:

```
Type (1-System, 2-User): 1

Process 2
Arrival Time: 0
Burst Time: 3
Type (1-System, 2-User): 1

Process 3
Arrival Time: 0
Burst Time: 8
Type (1-System, 2-User): 2

Process 4
Arrival Time: 10
Burst Time: 5
Type (1-System, 2-User): 1

PID     AT      BT      CT      TAT     WT
P1      0       4       6       6       2
P2      0       3       7       7       4
P3      0       8       20      20      12
P4      10      5       15      5       0

Average Turnaround Time = 9.50
Average Waiting Time = 4.50
```

Write a C program to simulate Real-Time CPU Scheduling algorithms: (Anyone)

a) Rate- Monotonic

```
#include <stdio.h>
#include <math.h>
#define MAX 10
// GCD
int gcd(int a, int b) {
   while (b != 0) {
      int t = b;
      b = a % b;
      a = t;
   }
   return a;
}
// LCM
int lcm(int a, int b) {
   return (a * b) / gcd(a, b);
}
// LCM of all
int find_lcm(int arr[], int n) {
   int res = arr[0];
   for (int i = 1; i < n; i++) {
      res = lcm(res, arr[i]);
   }
   return res;
}
int main() {
   int n, burst[MAX], period[MAX];
```

```
printf("Enter the number of processes:");
scanf("%d", &n);
printf("Enter the CPU burst times:\n");
for (int i = 0; i < n; i++)
   scanf("%d", &burst[i]);
printf("Enter the time periods:\n");
for (int i = 0; i < n; i++)
   scanf("%d", &period[i]);
int hyper = find_lcm(period, n);
printf("LCM=%d\n\n", hyper);
// Display table
printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for (int i = 0; i < n; i++) {
   printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
}
// Utilization
double U = 0;
for (int i = 0; i < n; i++) {
   U += (double)burst[i] / period[i];
}
double bound = n * (pow(2, (double)1 / n) - 1);
printf("\n%f <= %f => %s\n", U, bound, (U <= bound) ? "true" : "false");
printf("Scheduling occurs for %d ms\n\n", hyper);
// Scheduling simulation
int time = 0;
int remaining[MAX] = {0};
while (time < hyper) {
```

```
// release tasks
for (int i = 0; i < n; i++) {
    if (time % period[i] == 0) {
        remaining[i] = burst[i];
    }
}
int selected = -1;
// select highest priority (smallest period)
for (int i = 0; i < n; i++) {
    if (remaining[i] > 0) {
        if (selected == -1 || period[i] < period[selected]) {
            selected = i;
        }
    }
}
if (selected != -1) {
    printf("%dms onwards: Process %d running\n", time, selected + 1);
    remaining[selected]--;
    // run until preemption or finish
    int next = time + 1;
    while (next < hyper) {
        int preempt = 0;
        for (int i = 0; i < n; i++) {
            if (next % period[i] == 0 && period[i] < period[selected]) {
                preempt = 1;
            }
        }
        if (remaining[selected] == 0 || preempt)
            break;
```

```
            remaining[selected]--;
            next++;
        }
        time = next;
    } else {
        printf("%dms onwards: CPU is idle\n", time);
        time++;
    }
  }
  return 0;
}
```

OUTPUT:

```
Enter the number of processes:2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID     Burst   Period
1       20      50
2       35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle
76ms onwards: CPU is idle
77ms onwards: CPU is idle
78ms onwards: CPU is idle
79ms onwards: CPU is idle
80ms onwards: CPU is idle
81ms onwards: CPU is idle
82ms onwards: CPU is idle
83ms onwards: CPU is idle
84ms onwards: CPU is idle
85ms onwards: CPU is idle
```

b) Earliest-deadline First

```
#include <stdio.h>
#define MAX 10

int main() {
    int n;
    int burst[MAX], period[MAX], deadline[MAX];
    int remaining[MAX];
    printf("Enter number of tasks: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter burst, period, deadline for Task %d: ", i + 1);
        scanf("%d %d %d", &burst[i], &period[i], &deadline[i]);
        remaining[i] = burst[i];
    }

    int time = 0;
    int total_time = 20; // fixed like screenshot
    printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t\t%d\n", i + 1, burst[i], deadline[i], period[i]);
    }
    printf("\nScheduling occurs for %d ms\n\n", total_time);

    while (time < total_time) {
        // Reset jobs periodically (IMPORTANT for matching output)
        for (int i = 0; i < n; i++) {
            if (time % period[i] == 0) {
```

```
            remaining[i] = burst[i];
        }
    }
    int selected = -1;
    int min_deadline = 9999;

    // Select task with smallest FIXED deadline (not absolute)
    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            if (deadline[i] < min_deadline) {
                min_deadline = deadline[i];
                selected = i;
            }
        }
    }

    if (selected == -1) {
        printf("%dms : CPU is idle.\n", time);
    } else {
        printf("%dms : Task %d is running.\n", time, selected + 1);
        remaining[selected]--;
    }
    time++;
  }
  return 0;
}
```

OUTPUT:

```
Enter number of tasks: 3
Enter burst, period, deadline for Task 1: 2 5 4
Enter burst, period, deadline for Task 2: 3 20 7
Enter burst, period, deadline for Task 3: 2 10 8

PID     Burst   Deadline        Period
1       2       4               5
2       3       7               20
3       2       8               10

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 1 is running.
6ms : Task 1 is running.
7ms : Task 3 is running.
8ms : Task 3 is running.
9ms : CPU is idle.
10ms : Task 1 is running.
11ms : Task 1 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms : CPU is idle.
15ms : Task 1 is running.
16ms : Task 1 is running.
17ms : CPU is idle.
```